

BONA FIDE on Algorand: 9-contract constitutional architecture

The BONA FIDE constitutional reputation protocol maps cleanly to Algorand's native primitives with material advantages over EVM in every contract. **Algorand's native clawback makes Censura sanctions zero-code enforcement, box storage gives Tabularium unbounded records at \$0.00044/KB, and \$0.00011 transaction fees make Fides score updates and x402 API metering economically rational at sub-cent granularity.** The entire 9-contract system deploys as stateful ARC-4 applications in algopy (Algorand Python [Algorand](#) 5.0+), compiled by PuyaPy, [GitHub](#) tested via [algorand-python-testing](#) [PyPI](#) + AlgoKit LocalNet, and deployed through AlgoKit 3.0's idempotent [appDeployer](#). [Algorand](#) x402 on Algorand is production-ready [Algorand](#) as of February 2026, with GoPlausible operating the reference facilitator [Genfinity](#) [algorand](#) and full TypeScript/Python SDKs available. This plan delivers a 1:1 mirror of the Solidity system with a topologically ordered deployment sequence, BANKON AlgoIDNFT sovereign identity binding, Parsec Wallet signing flows, and x402 metering for the pythai.net API ecosystem.

The 9th contract is Aerarium — the constitutional treasury

The existing 8 contracts cover identity (Genius), currency (BonaToken), records (Tabularium), trust scoring (Fides), agreements (SponsioPactum), enforcement (Censura), governance (Senatus), and membership (Tessera). The critical functional gap is a **treasury** — the institution that pools, allocates, and disburses collective funds. BonaToken is a unit of account; Aerarium is the institution that manages wealth. This distinction was fundamental to Roman public finance.

The historical Aerarium Saturni sat in vaults beneath the Temple of Saturn, physically adjacent to the Tabularium in the Forum. The Romans stored financial assets and state documents in the same complex — laws did not become legally valid until deposited in the Aerarium. [Wikipedia](#) Managed by quaestors under Senate supervision, it divided into the common treasury (*aerarium Saturni*), a sacred reserve (*aerarium sanctius*, sealed except in existential crisis), and a military fund (*aerarium militare*). This maps directly to a protocol treasury contract managing fee collection, reward distribution, reserve management, and funding allocation governed by Senatus.

The Dii Novensiles — nine Etruscan gods with authority over thunderbolts — required collective deliberation even from Jupiter for certain destructive actions. [Wikipedia +2](#) Each had sovereign domain over one function; collectively they formed a complete enforcement council. [Grokipedia](#) Nine was the Roman number of institutional completeness.

► **This is an inference. Gregory should confirm whether Aerarium is correct or substitute another name.** Comitia (popular assembly, providing democratic counterweight to Senatus) and

Cursus (career progression ladder, institutionalizing the *cursus honorum* as a reputation tier system) were the next two candidates considered.

1. Genius — the soul-identity primitive

Civic meaning: *Genius loci* — the indwelling spirit of a person or place. Every Roman man had a Genius; every place had a genius loci. In BONA FIDE, each DAIO agent has a Genius contract instance as its on-chain soul.

Solidity function: Creates and stores the identity primitive for each agent — an on-chain record binding an address to an agent identity with metadata (name, type, creation timestamp, status).

Algorand implementation: Stateful ARC-4 application with **box storage** for per-agent identity records. Each Genius record is a box keyed by the agent's Algorand address (32 bytes), containing identity metadata (type, creation block, status flags, AlgoIDNFT ASA ID binding). Global state holds the contract admin, total agent count, and the BANKON AlgoIDNFT reference app/ASA. The application address serves as the canonical Genius registry — agents are "born" via an `inscribe_genius` ABI method that creates their identity box and optionally binds to a pre-minted AlgoIDNFT.

Why Algorand is superior: Box storage eliminates the EVM pattern of unbounded mappings that inflate state rent. Each Genius record's MBR is precisely calculable: a 256-byte identity record with a 32-byte key costs $(2,500 + 400 \times (32 + 256) = 117,700 \text{ microAlgo} \approx 0.118 \text{ ALGO } (\$0.013))$. No user opt-in required — the contract pays MBR from its own balance, unlike EVM where each mapping entry contributes to invisible state bloat. Rekeying an agent's operational account to the Genius application address enables key rotation without losing identity, directly mirroring the Roman concept that the Genius persists while the mortal form changes.

ABI interface:

```
inscribe_genius(agent: address, agent_type: uint8, metadata_hash: byte[32]) →  
uint64 // returns genius_id  
read_genius(agent: address) → (uint64, uint8, uint64, byte[32]) // id, type,  
birth_block, metadata  
bind_identity_nft(agent: address, nft_asa_id: uint64) → void  
revoke_genius(agent: address) → void // admin only; marks inactive  
is_active(agent: address) → bool
```

Storage model:

- Global state (4 keys): `admin` (address), `total_agents` (uint64), `algotxnft_app_id` (uint64), `version` (uint64)
- Boxes: key = agent address (32 bytes), value = packed struct (256 bytes): `genius_id` (8) + `agent_type` (1) + `birth_block` (8) + `status` (1) + `nft_asa_id` (8) + `metadata_hash` (32) + `reserved` (198)
- No local state required

Dependencies: Called by Tesseract (membership verification), Fides (identity lookup), Censura (revocation). Depends on BANKON AlgoIDNFT for identity binding.

Deployment order: 1st — foundational. All other contracts reference Genius.

2. BonaToken — the reputation credit token

Civic meaning: *Bona* — goods, property, wealth. *Bona fides* = good faith. BonaToken is the unit of civic credit — reputation made fungible.

Solidity function: ERC-20 token representing reputation/credit within the BONA FIDE system. Mintable by authorized contracts (Fides rewards), burnable by Censura (sanctions), transferable between agents.

Algorand implementation: Algorand Standard Asset (ASA) with **default-frozen = true** and all four role addresses (Manager, Freeze, Clawback) set to a controlling smart contract application address, following the **ARC-20 (Smart ASA)** pattern. The controlling application implements transfer logic, mint/burn authorization, and compliance checks. Since the ASA is default-frozen, **every transfer must route through the controlling contract's inner transactions** — the contract acts as the sole clawback authority, issuing `AssetTransfer` inner transactions with `asset_sender` (clawback field) set to the source holder.

Why Algorand is superior: The ARC-20 Smart ASA pattern provides **native programmable asset control** that EVM achieves only through custom ERC-20 logic. Clawback is a protocol-level primitive, not application code — the AVM enforces that frozen assets can only move via the designated clawback address. This means Censura sanctions are enforced at the **consensus layer**, not the application layer. Burn is implemented as clawback-to-reserve. Mint is transfer-from-reserve. Total supply management is intrinsic to the ASA, not a contract variable. Transaction fee for any BonaToken transfer: **\$0.00011** (vs. ~\$0.30 on Ethereum L1 for an ERC-20 transfer).

ABI interface (controlling application):

```

create_bona_token(total_supply: uint64, decimals: uint32) → uint64 // returns ASA
ID
transfer_bona(from: address, to: address, amount: uint64, memo: byte[]) → void
mint_bona(to: address, amount: uint64, reason: byte[32]) → void // Fides-
authorized
burn_bona(from: address, amount: uint64, reason: byte[32]) → void // Censura-
authorized
balance_of(agent: address) → uint64
authorize_minter(app_id: uint64) → void // admin: register Fides as minter
authorize_burner(app_id: uint64) → void // admin: register Censura as burner

```

Storage model:

- Global state (6 keys): `bona_asa_id` (uint64), `admin` (address), `total_minted` (uint64), `total_burned` (uint64), `fides_app_id` (uint64), `censura_app_id` (uint64)
- Boxes: `minters` (list of authorized app IDs), `burners` (list of authorized app IDs)
- No local state — balances tracked by the ASA ledger itself (L1 primitive)

Dependencies: Minting authorized by Fides. Burning authorized by Censura. Referenced by Senatus (voting weight), Aerarium (treasury deposits).

Deployment order: 2nd — required by Fides, Censura, Senatus, Aerarium.

3. Tabularium — the public record archive

Civic meaning: The *Tabularium* was the official records office of the Roman Republic, built 78 BC on the slopes of the Capitoline Hill overlooking the Forum. It stored *tabulae publicae* — bronze tablets inscribed with laws, treaties, census records, and financial accounts. It was the constitutional memory of the state.

Solidity function: On-chain ledger of civic acts — every significant action (reputation event, governance vote, sanction, pledge) is recorded as a timestamped, typed entry. The immutable public record that Fides reads to compute trust scores.

Algorand implementation: Stateful ARC-4 application using **box storage exclusively** for record entries. Each record is a box keyed by a sequential record ID (8-byte uint64), containing the act type, actor address, timestamp, referenced contract, and data hash. Global state tracks the current record count and authorized writer contracts. Box storage is ideal because the Tabularium is append-only with unbounded growth — exactly the access pattern boxes were designed for.

Why Algorand is superior: EVM stores mapping entries in contract storage slots at ~20,000 gas per SSTORE (~\$6 at current Ethereum prices per record). Algorand box storage costs $\frac{2,500 + 400 \times (8 + 128)}{1,000,000} = 56,900 \text{ microAlgo} \approx 0.057 \text{ ALGO} (\$0.006)$ per 128-byte record — and this MBR is **recoverable** if records are ever archived off-chain. Box records are directly addressable by key without iteration, and box references in transaction groups provide **1,024 bytes of read budget per reference** (8 references per call = 8KB readable per transaction). For a record archive, this is transformative: the Tabularium can serve as a verifiable audit trail at **\$0.006 per record** with instant random access.

ABI interface:

```
record_act(act_type: uint8, actor: address, ref_contract: uint64, data_hash:
byte[32]) → uint64 // returns record_id
read_record(record_id: uint64) → (uint8, address, uint64, uint64, byte[32]) //
type, actor, timestamp, ref, hash
record_count() → uint64
authorize_writer(app_id: uint64) → void // admin: register contracts that may
write records
is_authorized_writer(app_id: uint64) → bool
```

Storage model:

- Global state (4 keys): `record_count` (uint64), `admin` (address), `genius_app_id` (uint64), `version` (uint64)
- Boxes: key = record_id as 8-byte big-endian uint64, value = 128-byte packed struct: act_type (1) + actor (32) + timestamp (8) + ref_contract_app_id (8) + data_hash (32) + reserved (47)
- Additional box: `writers` — packed array of authorized writer app IDs

Dependencies: Written to by Fides, SponsioPactum, Censura, Senatus, Aerarium. Read by Fides for score computation.

Deployment order: 3rd — must exist before Fides (which reads it) and before any contract that writes records.

4. Fides — the trust and reputation scoring engine

Civic meaning: *Fides* was the Roman goddess of trust, good faith, and reliability. Her temple on the Capitoline was among Rome's oldest. *Bona fides* (good faith) was the foundation of Roman

contract law. In BONA FIDE, Fides computes a quantitative trust score from the accumulated record of civic acts.

Solidity function: Reads Tabularium entries for a given agent, applies a scoring algorithm (weighted by act type, recency, endorsements), and produces a Fides score. May mint BonaToken rewards for positive civic contributions.

Algorand implementation: Stateful ARC-4 application that reads Tabularium boxes via **inner application calls**, computes scores, and stores per-agent Fides scores in its own box storage. Score computation can be triggered by any party (permissionless) but writes are validated against Tabularium data. Mints BonaToken rewards via inner transaction to the BonaToken controlling application.

Why Algorand is superior: Inner transaction fee pooling means a Fides score update that reads 5 Tabularium records and mints a reward costs the caller a single pooled fee covering all inner transactions. At **\$0.00011 per inner transaction** (set to 0 fee, drawn from pool), a complex score recomputation touching 10 inner transactions costs the user ~\$0.0012 total. On EVM, the equivalent cross-contract calls with SLOAD operations could cost **\$1-5** in gas. The **256 inner transaction limit per call** (Algorand) means Fides can aggregate up to 256 Tabularium records in a single scoring pass, with a pooled opcode budget of up to **179,200 units** — sufficient for complex weighted averaging.

ABI interface:

```
compute_fides_score(agent: address, max_records: uint16) → uint64 // returns
score (0-10000 basis points)
read_fides_score(agent: address) → (uint64, uint64) // score, last_computed_round
mint_reward(agent: address, amount: uint64) → void // internal: calls BonaToken
mint
set_scoring_weights(weights: byte[64]) → void // admin: configure act-type
weights
get_scoring_weights() → byte[64]
```

Storage model:

- Global state (6 keys): `admin` (address), `tabularium_app_id` (uint64), `bonatoken_app_id` (uint64), `genius_app_id` (uint64), `scoring_weights` (bytes, 64B packed), `version` (uint64)
- Boxes: key = agent address (32 bytes), value = 64-byte score record: `fides_score` (8) + `last_computed_round` (8) + `total_acts_scored` (8) + `positive_acts` (8) + `negative_acts` (8) + `reserved` (24)

Dependencies: Reads Tabularium (inner app call). Calls BonaToken mint (inner txn).

Referenced by Senatus (voting weight lookup), Censura (threshold checks), Tessera (standing verification).

Deployment order: 4th — requires Tabularium and BonaToken app IDs.

5. SponsioPactum — the formal promise registry

Civic meaning: *Sponsio* was the most solemn form of verbal contract in Roman law — a formal oral promise invoking the gods as witnesses. *Pactum* was a binding agreement. The *sponsio* was the foundation of Roman obligation law (*obligatio*). SponsioPactum registers binding pledges between agents.

Solidity function: Registry of formal agreements between two or more agents — escrow conditions, deliverables, deadlines, dispute resolution terms. Each pactum has a lifecycle (proposed → accepted → fulfilled/breached → resolved).

Algorand implementation: Stateful ARC-4 application with box storage for each pactum. Each pactum box stores the parties, terms hash, escrow amounts (ALGO or BonaToken), status, and deadlines. **Atomic transaction groups** bind the pactum creation with escrow deposits — the pactum is created and funded in a single atomic group, preventing partial execution. Fulfillment and breach resolution trigger Tabularium record entries via inner transactions.

Why Algorand is superior: Algorand's **native atomic transaction grouping** (up to 16 transactions per group) is purpose-built for escrow patterns. A pactum creation + escrow deposit + Tabularium recording can be a single atomic group — either all succeed or none do. EVM requires custom reentrancy-guarded escrow logic. Inner transactions with fee pooling mean the contract can record to Tabularium and update Fides scores as part of fulfillment — at marginal cost of **\$0.00011 per inner transaction**. (Algorand)

ABI interface:

```

propose_pactum(counterparty: address, terms_hash: byte[32], escrow_algo: uint64,
escrow_bona: uint64, deadline: uint64) → uint64 // returns pactum_id
accept_pactum(pactum_id: uint64) → void // counterparty signs; escrow locked
fulfill_pactum(pactum_id: uint64, proof_hash: byte[32]) → void // initiator
claims fulfillment
confirm_fulfillment(pactum_id: uint64) → void // counterparty confirms → releases
escrow
dispute_pactum(pactum_id: uint64, evidence_hash: byte[32]) → void // triggers
dispute
read_pactum(pactum_id: uint64) → (address, address, byte[32], uint8, uint64,
uint64)

```

Storage model:

- Global state (5 keys): `admin` (address), `pactum_count` (uint64), `tabularium_app_id` (uint64), `genius_app_id` (uint64), `version` (uint64)
- Boxes: key = pactum_id (8 bytes), value = 256-byte struct: proposer (32) + counterparty (32) + terms_hash (32) + status (1) + escrow_algo (8) + escrow_bona (8) + deadline_round (8) + created_round (8) + proof_hash (32) + reserved (95)

Dependencies: Writes to Tabularium (inner app call). Reads Genius (identity verification). Escrow may hold BonaToken (inner ASA transfer). Disputes may escalate to Senatus.

Deployment order: 5th — requires Tabularium, Genius, BonaToken.

6. Censura — the censor and enforcement contract

Civic meaning: The *Censores* were among Rome's most powerful magistrates — they conducted the census, assessed citizens' moral fitness, controlled the Senate rolls, and could apply the *nota censoria* (mark of disgrace) that stripped civic standing. Censura removes bad actors and enforces civic standards.

Solidity function: Applies sanctions (token slashing, membership revocation, activity suspension) to agents who breach civic standards. Executes penalty logic triggered by governance decisions or automated threshold violations.

Algorand implementation: Stateful ARC-4 application that serves as the **enforcement nexus** of the entire system. Censura's application address is set as an authorized burner on BonaToken and has admin privileges on Genius (to revoke identity) and Tessera (to revoke membership). Sanctions are executed via **inner transactions**: BonaToken burn (clawback to reserve), Genius

status update (inner app call), Tessera revocation (inner app call), and Tabularium recording of the sanction.

Why Algorand is superior: This is where Algorand's **native clawback** mechanism delivers its most powerful advantage. On EVM, slashing tokens requires the sanctioned user to have approved the contract (which a bad actor would never do voluntarily), or the token contract must have custom admin-burn logic. On Algorand, because BonaToken is a **default-frozen ASA with Censura's controlling app as clawback**, the clawback can forcibly transfer tokens from any holder without approval — this is enforced at the **AVM consensus layer**, not application code. The sanctioned agent cannot prevent clawback. This is architecturally identical to the Roman Censor's authority to unilaterally strip a citizen's standing.

ABI interface:

```
apply_sanction(agent: address, sanction_type: uint8, severity: uint64,
evidence_hash: byte[32]) → uint64 // returns sanction_id
slash_bona(agent: address, amount: uint64) → void // inner txn: clawback
BonaToken to reserve
revoke_tessera(agent: address) → void // inner app call to Tessera
suspend_genius(agent: address, duration: uint64) → void // inner app call to
Genius
lift_sanction(sanction_id: uint64) → void // governance-authorized
read_sanction(sanction_id: uint64) → (address, uint8, uint64, uint64, byte[32])
is_sanctioned(agent: address) → bool
```

Storage model:

- Global state (8 keys): `admin` (address), `genius_app_id` (uint64), `bonatoken_app_id` (uint64), `tessera_app_id` (uint64), `tabularium_app_id` (uint64), `senatus_app_id` (uint64), `sanction_count` (uint64), `version` (uint64)
- Boxes: key = `sanction_id` (8 bytes), value = 128-byte struct. Also per-agent box: key = agent address, value = active sanction flags + expiry

Dependencies: Calls BonaToken (burn via clawback inner txn), Genius (suspend), Tessera (revoke), Tabularium (record). Authorized by Senatus governance decisions.

Deployment order: 8th — requires all other contract app IDs except Aerarium.

7. Senatus — the governance body

Civic meaning: The *Senatus* (from *senex*, "old man/elder") was Rome's supreme deliberative

body — 300+ senators who advised magistrates, controlled foreign policy, managed the Aerarium, and issued *senatus consulta* (decrees). In BONA FIDE, Senatus is weighted governance where voting power scales with Fides score.

Solidity function: Proposal creation, deliberation, and weighted voting. Voting weight determined by Fides score. Proposals can trigger actions on any other contract (parameter changes, sanctions, treasury allocations).

Algorand implementation: Stateful ARC-4 application with box storage for proposals and per-agent voting records. Proposal creation requires a minimum Fides score (checked via inner app call to Fides). Voting weight is the voter's Fides score at proposal creation time (snapshot model — score read once and stored in the proposal box to prevent mid-vote manipulation). Execution of passed proposals triggers inner transactions to the target contract.

Why Algorand is superior: Atomic grouping ensures vote-cast + Tabularium-record happens atomically. Fee pooling means a governance vote that reads Fides score, updates the proposal tally, and records to Tabularium costs a single outer fee (~\$0.0005 for a 5-inner-txn group). EVM governance contracts (Governor, Compound) require voters to pay **\$5-50 in gas per vote** on Ethereum L1, fundamentally biasing governance toward large holders. At **\$0.0005 per vote**, BONA FIDE on Algorand enables genuine small-stakeholder participation.

ABI interface:

```
create_proposal(title_hash: byte[32], body_hash: byte[32], target_app: uint64,
target_method: byte[4], target_args: byte[]) → uint64
cast_vote(proposal_id: uint64, vote: uint8) → void // 0=nay, 1=yea, 2=abstain;
weight = Fides score
execute_proposal(proposal_id: uint64) → void // if quorum + threshold met
read_proposal(proposal_id: uint64) → (byte[32], uint64, uint64, uint64, uint8) //
hash, yeas, nays, deadline, status
get_vote(proposal_id: uint64, voter: address) → (uint8, uint64) // vote, weight
```

Storage model:

- Global state (7 keys): `admin` (address), `fides_app_id` (uint64), `bonatoken_app_id` (uint64), `tabularium_app_id` (uint64), `quorum_threshold` (uint64), `proposal_count` (uint64), `version` (uint64)
- Boxes: proposal box keyed by `proposal_id` (8 bytes), value = 512 bytes (title_hash, body_hash, target info, vote tallies, deadline, status, snapshot data). Vote boxes keyed by `proposal_id + voter address` (40 bytes), value = 16 bytes (vote + weight).

Dependencies: Reads Fides (voter weight). Executes against any contract (inner app call). Writes Tabularium. Controls Censura authorization.

Deployment order: 6th — requires Fides, BonaToken, Tabularium.

8. Tessera — agent credential and membership token

Civic meaning: A *tessera hospitalis* was a token of friendship and mutual obligation, split between two parties — a physical proof of standing and alliance. Roman soldiers also received *tesserae* as duty tokens. In BONA FIDE, Tessera is proof of good standing — a verifiable credential of active membership.

Solidity function: Issues and manages membership credentials. A Tessera grants access to system functions (voting, pactum creation, reward claiming). Can be revoked by Censura or expired by time. Functions as a non-transferable soulbound token.

Algorand implementation: ARC-72 smart contract NFT — not an ASA. ARC-72 is specifically advantageous here because: (1) no opt-in required for recipients (unlike ASAs), eliminating friction for new members; (Algorand) (2) transfer restrictions are enforced in contract logic (soulbound = `arc72_transferFrom` reverts unless caller is admin); (3) credential metadata (standing level, expiry, permissions) is stored in contract boxes and queryable via `arc72_tokenURI`. Each Tessera is a unique NFT with `tokenId = genius_id`, binding it 1:1 to a Genius identity.

Why Algorand is superior: ARC-72 eliminates the ASA opt-in friction that plagues Algorand UX. On EVM, soulbound tokens (ERC-5192/ERC-6239) require custom transfer hooks that can be bypassed via delegate calls. ARC-72's smart contract enforcement means **the transfer restriction is the contract itself** — there is no underlying EVM `transferFrom` to bypass. Additionally, the Tessera contract can check Fides score before issuance (inner app call) and automatically expire credentials based on round number — something that requires external keepers on EVM.

ABI interface (extends ARC-72):

```

// ARC-72 base
arc72_ownerOf(token_id: uint256) → address
arc72_transferFrom(from: address, to: address, token_id: uint256) → void //
reverts: soulbound

// Tessera-specific
mint_tessera(agent: address) → uint256 // returns token_id = genius_id; checks
Fides threshold
revoke_tessera(agent: address) → void // Censura-authorized
renew_tessera(agent: address) → void // extends expiry if Fides score above
threshold
read_tessera(agent: address) → (uint256, uint64, uint64, uint8) // id,
issued_round, expiry_round, standing_level
is_member_in_good_standing(agent: address) → bool
arc72_tokenURI(token_id: uint256) → byte[256] // ARC-19 IPFS pointer to
credential metadata

```

Storage model:

- Global state (6 keys): `admin` (address), `genius_app_id` (uint64), `fides_app_id` (uint64), `censura_app_id` (uint64), `total_issued` (uint64), `version` (uint64)
- Boxes: key = agent address (32 bytes), value = 96-byte credential: token_id (32, uint256) + issued_round (8) + expiry_round (8) + standing_level (1) + permissions_bitmask (8) + metadata_uri_hash (32) + reserved (7)

Dependencies: Reads Genius (identity binding), Fides (standing check). Called by Censura (revocation). Checked by SponsioPactum, Senatus (membership gate).

Deployment order: 7th — requires Genius, Fides, and must exist before Censura is wired.

9. Aerarium — the constitutional treasury ▀ INFERRED

Civic meaning: *Aerarium* (from *aes*, bronze/money) — the central public treasury of the Roman Republic. [Wikipedia](#) Housed in the Temple of Saturn [Encyclopedia Britannica](#) adjacent to the Tabularium. Managed by quaestors under Senate supervision. [En-academic](#) Divided into common treasury (*Saturni*), sacred reserve (*sanctius*), and military fund (*militare*). [IFAC](#) Laws were not valid until deposited in the Aerarium. [Wikipedia](#) It was the financial sovereignty of the *res publica*.

Solidity function: Protocol treasury — collects fees from system operations (pactum fees, identity registration fees, x402 API revenue), manages staking deposits, distributes rewards,

funds governance-approved expenditures. The financial backbone that gives Senatus fiscal authority.

Algorand implementation: Stateful ARC-4 application managing the protocol's ALGO and BonaToken reserves. The application account holds pooled funds. **Inner transactions** handle all disbursements — reward payouts, bounty payments, fee distributions. (Algorand) Box storage tracks allocation categories (mirroring *aerarium Saturni* vs *sanctius*): operational fund, reserve fund, and reward pool with separate accounting. Senatus proposals can authorize Aerarium withdrawals via inner app calls. Fee collection happens via inner transactions from other contracts (e.g., SponsioPactum takes a fee on pactum creation and forwards to Aerarium).

Why Algorand is superior: Algorand's **inner transaction model** makes the Aerarium pattern clean: the contract holds funds in its application account and disburses via inner (Payment) or (AssetTransfer) transactions — no approve/transferFrom dance. The minimum balance requirement (**0.1 ALGO base**) on the application account ensures it always has funds to exist. Fee collection via inner transactions from other contracts is **zero marginal cost** (fee-pooled). (Algorand) On EVM, treasury contracts require complex approve → transferFrom flows and are frequent targets of reentrancy attacks. Algorand's single-round finality means treasury operations settle immediately.

ABI interface:

```
deposit_algo(amount: uint64, category: uint8) → void // category: 0=operational,
1=reserve, 2=reward
deposit_bona(amount: uint64, category: uint8) → void
withdraw_algo(recipient: address, amount: uint64, proposal_id: uint64) → void //
Senatus-authorized
withdraw_bona(recipient: address, amount: uint64, proposal_id: uint64) → void
distribute_rewards(recipients: byte[], amounts: byte[]) → void // batch inner
txns
read_balance(category: uint8) → (uint64, uint64) // algo_balance, bona_balance
read_total_reserves() → (uint64, uint64)
```

Storage model:

- Global state (8 keys): (admin) (address), (senatus_app_id) (uint64), (bonatoken_app_id) (uint64), (tabularium_app_id) (uint64), (operational_algo) (uint64), (reserve_algo) (uint64), (reward_algo) (uint64), (version) (uint64)
- Boxes: (allocation_log) — sequential log of all deposits/withdrawals; (category_bona_balances) — BonaToken balances per category

Dependencies: Authorized by Senatus (withdrawal approval). Calls BonaToken (inner ASA transfer). Writes to Tabularium (financial records). Receives from SponsioPactum (fees), Fides (reward funding).

Deployment order: 9th (last) — requires Senatus, BonaToken, Tabularium.

BANKON AlgoIDNFT sovereign identity binding

BANKON AlgoIDNFT implements sovereign agent identity as a **1-of-1 ASA with total supply = 1, decimals = 0, and default-frozen = true**, with metadata via **ARC-19 IPFS pointer**. The existing Bankon Wallet (EVM-side) uses Alchemy Account Kit with smart accounts and WebAuthn signers for Google/Apple/email authentication. The Algorand port must map this to native Algorand identity.

Minting flow: The AlgoIDNFT is minted by the Genius contract via inner `AssetConfig` transaction (creating a new ASA). The ASA's Reserve address encodes the IPFS CID of the agent's identity metadata (per ARC-19: `template-ipfs://{ipfscid:1:dag-pb:reserve:sha2-256}/identity.json`). The Manager address is set to the Genius contract's application address (enabling metadata updates). Freeze and Clawback addresses are set to the Genius application (enabling identity control). Total supply = 1, default-frozen = true, decimals = 0. The agent opts in and receives the single unit. This creates a **non-transferable, contract-controlled 1-of-1 identity NFT** bound to the agent's Genius record.

Genius binding: The Genius box for the agent stores the `nft_asa_id` field pointing to this ASA. The AlgoIDNFT's ARC-19 metadata JSON contains the `genius_id`, creating a bidirectional binding. The Genius contract is the sole authority over the NFT's existence (can destroy it on identity revocation).

Key rotation via rekeying: The agent's Algorand account can be **rekeyed** to a new authorized address without changing the account's public address. Since the AlgoIDNFT is held by the account address (not the authorized key), rekeying preserves the identity NFT's ownership while rotating the operational signing key. The Genius contract validates rekeying operations: the `rotate_key` method accepts a new authorized address, verifies the caller is the current authorized address, and records the rotation in Tabularium. The critical security check: `assert Txn.rekey_to == Global.zero_address` in all non-rotation methods to prevent unauthorized rekeying.

Parsec Wallet integration architecture

Parsec Wallet (parsec-wallet GitHub organization, in-house wallet for the DELTAVERSE

ecosystem) must expose the following for BONA FIDE transaction signing. Based on the current WalletConnect v2 ecosystem on Algorand and the ARC-1/ARC-25 standards:

Required signing capabilities: Parsec must implement the **WalletConnect v2 protocol** using Algorand's CAIP-2 chain identifiers (MainNet: `algorand:SG01GKSzyE7IEPItTxCByw9x8FmnrCDe`, TestNet: `algorand:testnet-v1.0`). The wallet exposes `algo_signTxn` JSON-RPC method per ARC-1, accepting `WalletTransaction[]` arrays containing base64-encoded unsigned transactions. For BONA FIDE, the wallet must handle these specific transaction patterns:

- **ASA opt-in signing:** Single `AssetTransfer` where `sender == receiver`, amount = 0, for BonaToken and AlgoIDNFT opt-in. Display: "Opt in to BONA Token (ASA #XXXX)"
- **Atomic group signing:** Groups of 2-8 transactions for pactum creation (escrow deposit + app call), governance voting (app call + Tabularium record), and Fides score computation. Parsec must display the full group with human-readable method names decoded from ARC-4 selectors
- **Inner transaction awareness:** When the user signs an outer app call that triggers inner transactions (e.g., calling Fides which calls Tabularium), Parsec should display the expected inner transaction effects (readable via simulation using `algod /v2/transactions/simulate`)
- **Box read requests:** Parsec should support `algod` box read API calls (`/v2/applications/{app-id}/box?name=...`) for displaying Genius identity, Fides scores, Tessera status, and Tabularium records in the wallet UI

dApp-wallet communication: Standard WalletConnect v2 relay via `@walletconnect/modal-sign-html` on the dApp side. Parsec registers with a WalletConnect Cloud project ID. Deep link URI: `parsec-wallet://wc?uri={encoded_wc_uri}`. Parsec should integrate the **UseWallet** library abstractions for multi-wallet compatibility but prioritize its own native signing UX.

Implementation priority: Given Parsec Wallet's early-stage status (currently only the slingshot fork is public), the minimum viable integration is: (1) WalletConnect v2 session management, (2) `algo_signTxn` for individual and grouped transactions, (3) transaction simulation display, (4) ARC-4 method name decoding for human-readable confirmations.

x402 micropayment metering for the pythai.net API ecosystem

x402 on Algorand is **production-ready** — the specification was officially merged with Coinbase's x402 implementation on February 23, 2026. **GoPlausible** operates the reference Algorand facilitator at `https://facilitator.goplausible.xyz` with full OpenAPI documentation. SDKs

are available: `@x402-avm/*` for TypeScript (Express, Hono, Next.js, fetch, axios) and `x402-avm` on PyPI for Python (FastAPI, Flask, httpx, requests).

Implementation for `mindx.pythai.net` / `agenticplace.pythai.net` / `bankon.pythai.net`:

The API gateway wraps each endpoint with x402 middleware. When a client requests a metered endpoint without payment, the server returns HTTP 402 with a `PAYMENT-REQUIRED` header specifying: `scheme = exact`, `network = algorand:mainnet`, `amount` (in USDC ASA or ALGO), `payTo` (PYTHAI treasury address), and `maxTimeoutSeconds`. The client's wallet (Parsec or any x402-compatible agent) signs an Algorand payment transaction (ASA transfer for USDC, or ALGO payment) with the **note field containing the SHA-256 hash of the API request** (URL + parameters + nonce). The client retries with `PAYMENT-SIGNATURE` header. The server POSTs to the GoPlausible facilitator's `/verify` endpoint, which verifies the Algorand transaction signature, checks amount/recipient/nonce, and returns verification. On success, the server executes the request and returns the response with `PAYMENT-RESPONSE` header. The facilitator settles by broadcasting to the Algorand network — **instant finality in ~3.3 seconds**.

Fee analysis for sub-cent API metering: At ALGO price ~\$0.11 (April 2026), Algorand's minimum transaction fee of 0.001 ALGO costs **\$0.00011**. For a \$0.001/request API pricing tier: transaction overhead is 11%, leaving **89% to the merchant**. On Base (\$0.003/tx): the merchant loses money at \$0.001 pricing. On Solana (\$0.00025/tx): 75% to merchant. **Algorand is the only production chain where sub-cent API metering is economically viable with per-request settlement**. At \$0.01/request pricing, Algorand overhead drops to **1.1%** — comparable to credit card interchange but without the \$0.30 minimum that makes micropayments impossible on Stripe.

Spam protection: Algorand's minimum balance requirement (**0.1 ALGO per account**) acts as natural Sybil resistance. Creating a wallet to spam API requests costs \$0.011 plus 0.1 ALGO per ASA opt-in. Combined with x402's nonce-based replay protection and the facilitator's 120-second settlement cache, this provides multi-layer spam defense.

Server-side verification pattern (Python/FastAPI):

```
python
```



```

from x402_avm import PaymentMiddleware # x402-avm PyPI package

app = FastAPI()
x402 = PaymentMiddleware(
    facilitator="https://facilitator.goplausible.xyz",
    pay_to="PYTHAI_ALGORAND_ADDRESS",
    network="algorand:mainnet"
)

@app.get("/api/mindx/query")
@x402.require_payment(amount="1000", asset="USDC_ASA_ID") # 0.001 USDC
async def mindx_query(q: str): ...

```

AgenticPlace agent registry — ERC-8004 mapping to Algorand

ERC-8004 ("Trustless Agents") is a real Ethereum standard (Draft status, deployed on Ethereum mainnet January 29, 2026) defining three on-chain registries for AI agent identity, reputation, and validation. Authors include representatives from MetaMask, Ethereum Foundation, Google, and Coinbase. The standard's three registries map to BONA FIDE's existing contracts:

- **ERC-8004 Identity Registry** (ERC-721 + URIStorage) → **Genius + AlgoIDNFT** — each agent gets a unique NFT identity pointing to a structured JSON "agent card" with name, capabilities, service endpoints (MCP, A2A), and payment address. On Algorand, the Genius box + ARC-19 AlgoIDNFT already implements this. The agent card JSON at the ARC-19 IPFS URI should conform to ERC-8004's agent card schema for cross-ecosystem compatibility.
- **ERC-8004 Reputation Registry** (signed feedback with int128 scores) → **Fides + Tabularium** — public on-chain reputation scores derived from recorded civic acts. Fides scores can be exposed via an ARC-4 method that returns data in ERC-8004-compatible format for cross-chain queries via State Proofs.
- **ERC-8004 Validation Registry** (validator hooks) → **Censura + Tessera** — independent validation checks and enforcement. Tessera membership serves as the "validated agent" credential.

The AgenticPlace registry at agenticplace.pythai.net should expose a unified API that wraps Genius/Fides/Tessera queries and returns ERC-8004-compatible JSON agent cards, enabling BONA FIDE agents to interoperate with EVM-native ERC-8004 agent registries. **State Proofs** (production on Algorand since September 2022, generated every 256 rounds / ~12 minutes)

provide the cryptographic bridge — an Algorand agent's Fides score can be verified on Polygon/Ethereum via a State Proof light client without trusting a centralized bridge.

Deployment sequence — topological order with MBR funding

The deployment must respect dependency ordering. Each contract's `__init__` method wires in the app IDs of its dependencies.

1. **Genius** (no dependencies) → deploy, note `genius_app_id`. Fund application account: ~0.5 ALGO base MBR + box budget for initial agents
2. **BonaToken controller** (no dependencies at deploy; Genius wired post-deploy) → deploy, create BONA ASA via inner transaction, note `bonatoken_app_id` and `bona_asa_id`. Fund: 0.5 ALGO + 0.1 ALGO for ASA creation
3. **Tabularium** (depends on Genius for identity verification) → deploy, wire `genius_app_id`. Fund: 0.5 ALGO base + scale box budget for expected record volume (1,000 records × 0.057 ALGO = **57 ALGO initial box budget**)
4. **Fides** (depends on Tabularium, BonaToken, Genius) → deploy, wire `tabularium_app_id`, `bonatoken_app_id`, `genius_app_id`. Register Fides as authorized minter on BonaToken. Fund: 0.3 ALGO base + score box budget
5. **SponsioPactum** (depends on Tabularium, Genius, BonaToken) → deploy, wire deps. Register as Tabularium writer. Fund: 0.3 ALGO + pactum box budget
6. **Senatus** (depends on Fides, BonaToken, Tabularium) → deploy, wire deps. Register as Tabularium writer. Fund: 0.3 ALGO + proposal/vote box budget
7. **Tessera** (depends on Genius, Fides, needs Censura post-deploy) → deploy, wire `genius_app_id`, `fides_app_id`. Fund: 0.3 ALGO + credential box budget
8. **Censura** (depends on ALL: Genius, BonaToken, Tessera, Tabularium, Senatus) → deploy, wire all app IDs. Register as authorized burner on BonaToken. Register as authorized revoker on Tessera and Genius. Fund: 0.3 ALGO + sanction box budget
9. **Aerarium** (depends on Senatus, BonaToken, Tabularium) → deploy, wire deps. Register as Tabularium writer. Fund: initial treasury allocation (project-dependent)

Post-deployment wiring: After all 9 are deployed, execute admin calls to cross-register: wire `censura_app_id` into Tessera; wire `aerarium_app_id` into Senatus; register all writer contracts with Tabularium.

Total estimated MBR for TestNet deployment: ~65 ALGO (dominated by Tabularium box pre-funding). TestNet ALGO is free via `algokit dispenser fund`.

TestNet → MainNet migration: Deploy in same order on MainNet using `algokit project deploy mainnet` with `.env.mainnet` containing MainNet algod/indexer endpoints and funded deployer account. AlgoKit's idempotent `appDeployer` handles named app tracking across environments.

Testing strategy — AlgoKit + algopy + pytest

The recommended development and testing stack:

- **Language:** algopy (Algorand Python 5.0+), Python ≥ 3.12
- **Compiler:** PuyaPy (optimizing compiler, `puya` package)
- **Framework:** AlgoKit 3.0 CLI (`algokit init -t python`, Production preset)
- **Unit testing:** `algorand-python-testing` (PyPI) with pytest — offline AVM simulation, no running node required. Use `algopy_testing_context()` context manager for contract instantiation, state mocking, and inner transaction simulation
- **Integration testing:** `algokit localnet start` (Docker-based local Algorand network) + pytest fixtures deploying real contracts against LocalNet
- **Client generation:** `algokit generate client` from ARC-56 specs → typed Python clients for each contract
- **Explorer:** Lora (`algokit explore`) for LocalNet visualization; Allo.info or Pera Explorer for TestNet/MainNet

Foundry parallel: Gregory uses Foundry (Forge) for EVM. The Algorand equivalent is: `forge build` → `algokit compile`; `forge test` → `pytest` with `algorand-python-testing`; `forge script` → `algokit project deploy`; `cast call` → `algokit task send` or typed client method calls. The key difference: algopy unit tests run **offline** (no node required), unlike Forge which uses an embedded EVM — this makes algopy tests faster.

Beaker is deprecated (explicitly sunset in favor of algopy). PyTeal's last release was v0.27.0 (February 2025). Do not use either for new contracts.

What we build first — concrete next steps

Week 1: Scaffold and Genius

```
bash
```

```
algokit init -n bonafide-algorand -t python --defaults
cd bonafide-algorand
algokit localnet start
```

Create `smart_contracts/genius/contract.py` — implement `inscribe_genius`, `read_genius`, `bind_identity_nft`, `is_active`. Write unit tests with `algorand-python-testing`. Deploy to LocalNet. This is the foundation contract that everything else references.

Week 2: BonaToken + Tabularium Implement the BonaToken ARC-20 Smart ASA controller — ASA creation via inner transaction, transfer/mint/burn methods. Implement Tabularium with box-based record storage. Unit test both. Deploy to LocalNet. Wire BonaToken → authorize Tabularium as a dependency.

Week 3: Fides + AlgoIDNFT Implement Fides score computation with inner calls to Tabularium. Implement AlgoIDNFT minting within Genius (ARC-19 ASA creation). Integration test: create Genius → mint AlgoIDNFT → record acts in Tabularium → compute Fides score → mint BonaToken reward. This end-to-end flow validates the core reputation loop.

Week 4: SponsioPactum + Senatus + Tessera Implement remaining civic contracts. Integration test: create pactum → fulfill → record → update Fides → issue Tessera → create governance proposal → vote → execute.

Week 5: Censura + Aerarium + x402 integration Implement enforcement and treasury. Wire Censura with clawback authority. Set up x402 middleware on a test API endpoint using `@x402-avm/express` or `x402-avm` Python package with GoPlausible facilitator. End-to-end test: agent calls metered API → pays via x402 → payment recorded in Tabularium → Fides score updated.

Week 6: TestNet deployment + Parsec signing flows Deploy all 9 contracts to TestNet in topological order. Fund via `algokit dispenser fund`. Test full system with Parsec Wallet signing (WalletConnect v2 integration — even a minimal web-based signing UI is sufficient for TestNet validation). Verify all cross-contract calls, box storage, and ASA operations.

The first command to run:

```
bash

pip install algokit algorand-python algorand-python-testing
algokit localnet start
algokit init -n bonafide-algorand -t python
```

Build Genius first. Everything flows from identity.